

LangChain is an open-source framework for building LLM applications and agents. It gives you reusable building blocks for prompts, models, tools, retrievers, vector stores, middleware, and agent workflows. LangChain's current positioning is high-level application composition.

LangGraph is the lower-level runtime/orchestration layer for stateful, long-running agents and workflows. It models applications as **graphs** with shared **state**, **nodes**, and **edges**, and adds persistence, memory, streaming, debugging support, and human-in-the-loop control. LangChain's newer agent abstractions are built on top of LangGraph.

LangSmith is the observability/evaluation/deployment side. It helps you trace runs, inspect steps, evaluate outputs, compare prompts or models, monitor latency/cost/failures, and debug RAG/agent systems in production.

Main components of LangChain:

At a practical level, these are the parts you use most:

a) Models

This is the LLM or chat model that generates text or reasoning outputs.

b) Prompts

Instructions and templates that shape model behavior.

c) Output parsers / structured outputs

These turn raw model text into JSON, typed objects, tool calls, etc.

d) Retrievers

Components that fetch relevant documents from a knowledge source. LangChain defines a retriever as an interface that takes a string query and returns documents; it is broader than a vector store.

e) Vector stores

Storage + similarity search over embeddings.

f) Tools

External capabilities like search, calculator, SQL, APIs, file readers, web fetchers.

g) Agents

Systems where the model decides which tools to call and in what order.

h) Memory / state

Conversation history or persistent knowledge used across steps or sessions. LangGraph distinguishes short-term thread memory and longer-term memory systems.

i) Middleware / orchestration

Cross-cutting control such as retries, guardrails, tool policies, observability hooks, and runtime logic.

RAG = Retrieval-Augmented Generation.

Instead of asking the LLM to answer only from its training knowledge, you first retrieve relevant context from your own data, then pass that context into the model to produce the answer.

LangChain's docs describe both a simple two-step RAG chain and a more agentic RAG approach.

Core RAG flow

1. Ingest documents
2. Clean text
3. Chunk text

4. Create embeddings
5. Store embeddings in a vector store / vector DB
6. At query time, embed the user query
7. Retrieve similar chunks
8. Put retrieved chunks into a prompt template
9. Ask the LLM to answer based on that context

Document ingestion:

Load Data from PDF's, databases, Word docs, websites, emails, google drive.

Then normalize text:

- remove boilerplate, keep headings, preserve tables if possible.
- attach metadata such as source, page number, section, date, author, product, tenant, permissions.

Chunking:

Chunking means splitting a large document into smaller units before embedding. Pinecone describes chunking as breaking large text into smaller segments that are big enough to contain meaningful information but small enough for performant retrieval.

Why chunking is needed

If you embed a whole 100-page PDF as one vector:

- retrieval becomes too coarse
- the returned context is huge and noisy
- the model may miss the exact answer

If you chunk too small:

- meaning gets fragmented
- context is incomplete
- retrieval may return partial nonsense

Common chunking types

1) Fixed-size chunking

Split every N characters or N tokens.

Good:

- simple
- fast
- easy baseline

Bad:

- may cut a sentence or paragraph in the middle
- ignores document structure

2) Recursive chunking

Split by headings → paragraphs → sentences → words until chunk size fits.

Good:

- preserves semantics better
- common default choice

3) Sentence-based chunking

Keep sentences together up to a target size.

Good:

- better readability
- better semantic integrity

4) Paragraph-based chunking

Each chunk is a paragraph or group of nearby paragraphs.

Good for:

- policies
- FAQs
- manuals
- legal docs

5) Semantic chunking

Split where the meaning changes instead of by raw size.

Good:

- more coherent chunks

Bad:

- more complex
- sometimes slower or harder to tune

6) Structure-aware chunking

Use document structure:

- title
- headings
- tables
- sections
- code blocks
- bullet lists

Best for:

- manuals
- knowledge bases
- API docs
- research papers

Vector embeddings

An **embedding** is a dense numeric representation of meaning. Chroma's docs describe embeddings as numeric representations of data that capture meaning in a form AI systems can work with.

Why embeddings matter

They let you search by meaning, not just exact keywords.

Example:

- query: "refund policy"
- stored chunk: "customers can request a reimbursement within 30 days"

Keyword search may miss it.

Embedding search can still find it.

Different types of embeddings

1) Dense embeddings

Most common in RAG.

A chunk becomes a dense vector like [0.13, -0.84, ...].

Used for semantic similarity.

2) Sparse embeddings

Closer to keyword-style matching.

Useful for exact term importance and lexical signals.

3) Hybrid retrieval

Combine dense + sparse.

This is often stronger than either alone for production search.

Chroma explicitly supports dense and sparse retrieval/search workflows.

4) Text embeddings

For documents, questions, passages.

5) Multimodal embeddings

For text + image, sometimes audio/video.

Chroma notes embeddings can represent text, images, and eventually audio/video.

Embedding model considerations

When choosing an embedding model, evaluate:

- dimension size
- cost
- latency
- multilingual support
- domain fit
- retrieval accuracy
- whether document and query embedding spaces match

Query embedding vs document embedding

At ingestion time: embed document chunks

At runtime: embed the user query

Then compare them using similarity search

Vector storage

After chunking and embedding, you **store** vectors with metadata.

A stored record usually looks like:

- id
- embedding
- text chunk
- metadata:
 - source file
 - page number
 - section
 - timestamp
 - tenant/user
 - tags
 - permission scope

Pinecone's data modeling guidance emphasizes storing chunked documents with structured IDs and metadata so you can upsert, search, fetch, update, and delete cleanly.

Vector Database:

A **vector database** stores embeddings and supports fast nearest-neighbor search, usually with metadata filtering, CRUD, and scalability features. Pinecone defines vector databases as systems that index and store vector embeddings for fast retrieval and similarity search, with features like metadata filtering and horizontal scaling.

Types of vector storage systems

A) Library-based index

Example: **FAISS**

This is not a full managed database by itself. It is a high-performance similarity search library. FAISS documentation says it is a library for efficient similarity search and clustering of dense vectors, including very large sets that may not fit in RAM, with GPU support available.

Use when:

- local experimentation
- notebooks
- prototypes
- custom pipelines
- you manage persistence yourself

B) Embedded/local vector database

Example: **Chroma**

Useful for:

- local development
- lightweight deployments
- fast prototyping
- apps where you want easy setup

Chroma positions itself as AI data infrastructure with storage, metadata, dense/sparse search, and retrieval capabilities.

C) Managed cloud vector database

Example: **Pinecone**

Useful for:

- scalable production
- managed infrastructure
- operational simplicity
- high-availability search services

Pinecone is a dedicated managed vector database platform.

FAISS vs Chroma vs Pinecone

FAISS

What it is: a vector similarity search/indexing library.

Strengths

- very fast
- excellent for research/prototyping
- many index types
- GPU support
- great control over ANN tradeoffs

Limitations

- not a full managed DB by default

- you handle persistence / serving / scaling
- metadata filtering is not as natural as in managed vector DBs

Chroma

What it is: open-source AI retrieval/data infrastructure with built-in document + metadata storage and retrieval.

Strengths

- easy local setup
- good developer ergonomics
- handles retrieval-oriented workflows well
- good for prototypes and some production cases

Limitations

- not as infrastructure-heavy/managed as Pinecone
- large-scale operational needs may require more planning

Pinecone

What it is: managed vector database platform.

Strengths

- production-ready managed service
- metadata filtering
- scaling
- serverless/managed operational model
- good for enterprise workloads

Limitations

- cloud dependency
- cost
- less “run-everything-locally” friendly than FAISS/Chroma

Which one should to use

- Learning / prototype / local notebook → FAISS or Chroma
- Quick developer-friendly app → Chroma
- Production managed scale → Pinecone
- Custom low-level ANN research → FAISS

Retriever

LangChain defines a retriever as an interface that returns documents given an unstructured query, and says it is broader than a vector store because it only needs to retrieve, not store.

A retriever may use:

- vector similarity
- keyword search
- hybrid search
- metadata filters
- query rewriting
- external knowledge sources like Wikipedia or Kendra

Retriever may:

1. rewrite the query
2. search vector DB
3. filter only policy docs
4. return top 5 chunks

Then the LLM answers from those chunks.

Prompt template / output template

a) Prompt template

A reusable prompt structure.

Example structure:

- system instructions
- user question
- retrieved context
- answer format rules

For RAG, the prompt often looks like:

- You are a QA assistant
- Use only the provided context
- If answer is not present, say you don't know
- Context: {retrieved_chunks}
- Question: {user_query}

b) Output template / structured output

This forces the answer into a format like:

- JSON
- bullet summary
- final answer + citations
- classification labels
- SQL
- tool arguments

Why needed:

- consistency
- downstream parsing
- safer automation
- easier evaluation

What is query decomposition

Query decomposition means breaking one complex question into smaller sub-questions.

Why decomposition helps

- better retrieval for each sub-question
- less ambiguity
- better coverage
- less chance of missing details in one big query

Optimization techniques for storing and searching vector embeddings

This is where many RAG systems succeed or fail.

1) Better chunking

Most important first optimization.

2) Better metadata design

Store:

- source
- section
- date
- user/team/org
- language
- product/version
- permissions

Then filter before or during retrieval.

3) ANN indexing

Approximate nearest neighbor search makes retrieval fast at scale.

4) Index compression / quantization

FAISS includes quantization and different index structures to trade memory, speed, and accuracy. The FAISS docs and wiki emphasize multiple index types and parameter tuning for large-scale similarity search.

5) Reranking

Retrieve top 20 quickly, then rerank top candidates with a stronger model.

6) Hybrid retrieval

Dense + sparse/keyword together.

7) Query rewriting

Rephrase the user query before retrieval. LangChain documents a RePhraseQuery retriever for this exact purpose.

8) Parent-child retrieval

Store small child chunks for search, but return a larger parent section for final context.

9) Deduplication

Near-duplicate chunks hurt quality and waste index space.

10) Freshness strategy

Re-index updated docs incrementally.

11) Namespace / multitenancy partitioning

Separate embeddings by team, product, customer, or region.

12) Cache query embeddings / retrieval results

Useful for repeated questions.

Offline pipeline vs online pipeline

This is very important.

Offline pipeline

Runs before user asks a question.

What it does

- collect documents
- clean/normalize data
- chunk content
- generate embeddings
- store in vector DB
- build/update indexes

- attach metadata
- schedule refresh jobs

This is similar to data engineering ingestion/preprocessing.

Online pipeline

Runs when the user asks a question.

What it does

- receive user query
- maybe rewrite/decompose query
- embed query
- retrieve relevant chunks
- maybe rerank
- build final prompt
- call LLM
- return answer
- log traces/feedback

Memory

Memory is the mechanism that lets an LLM application keep relevant information across turns or across time. LangGraph's memory overview says memory is crucial because agents need to remember previous interactions, learn from feedback, and adapt to user preferences; it distinguishes short-term thread memory and longer-term memory.

Why memory is needed

Without memory:

- chatbot forgets previous turns
- agent repeats questions
- no personalization
- long tasks break apart
- multi-step workflows lose context

Types of memory

1) Short-term memory

Current conversation state.

Example:

- user already said their name
- user already picked a plan
- the last few tool outputs

2) Long-term memory

Information remembered across sessions.

Example:

- user preferences
- recurring workflows
- organization-specific facts

3) Episodic memory

Stores past interactions/events.

Example:

- "Last week this user asked about policy version 2.1"

4) Semantic memory

Stores facts/knowledge.

Example:

- “Company refund SLA is 3 business days”

5) Working memory

Temporary data used while solving one task.

Example:

- intermediate steps in a multi-hop reasoning pipeline

Tools:

A tool is an external capability the model can call.

Examples:

- web search
- calculator
- SQL database
- Python execution
- email sender
- CRM lookup
- internal API
- vector retriever

Why tools are needed

LLMs alone are limited:

- may hallucinate
- don't know real-time data
- can't access your systems by themselves
- can't reliably compute everything
- can't take actions without tool access

Token limits

A **token limit** is the maximum context length a model can process in one request. This includes:

- system prompt
- user prompt
- retrieved documents
- tool results
- prior conversation history
- the model's generated answer budget

Why token limits matter in RAG

If you retrieve too much context:

- prompt becomes expensive
- latency increases
- the model may get distracted
- some context may be truncated

Types of RAG:

1) Naive RAG

The basic pipeline:

- chunk
- embed
- store
- retrieve top-k
- prompt LLM

Good for

- first prototype
- simple FAQ bots

Weakness

- poor with ambiguity, multi-hop queries, noise

2) Two-step RAG

LangChain explicitly describes a two-step RAG chain with a single LLM call after retrieval.

Flow

1. Retrieve
2. Generate answer

Good for

- speed
- predictable architecture
- easy evaluation

Weakness

- limited adaptability for hard questions

3) Agentic RAG

LangChain also describes a RAG agent that uses tools/search in a more dynamic way.

Flow

The agent decides:

- whether to retrieve
- what retriever/tool to use
- whether to search again
- whether to reformulate the question

Good for

- open-ended tasks
- multi-step knowledge work
- deeper research

Weakness

- more latency
- more complexity
- harder debugging unless you use observability

4) Multi-query RAG

Generate several reformulations of the user question, retrieve for each, then merge results.

Why useful

One wording may miss good documents.

Several wordings improve recall.

Good for

- ambiguous questions
- terminology mismatch
- synonym-heavy domains

5) Query decomposition RAG

Break one complex question into sub-questions, retrieve separately, then synthesize.

Good for

- comparison
- multi-hop reasoning
- long analytical questions

Example

“Compare policy changes between two years and explain state impact.”

6) Hybrid RAG

Use dense vector search + sparse keyword/BM25 search together.

Good for

- exact terms + semantic meaning
- legal/product/code/doc search
- names, IDs, clauses, and fuzzy meaning together

Very strong in practice

Many production systems use hybrid retrieval.